

Relatório de Auditoria de Segurança

Screen Share

Data: 31/08/2026

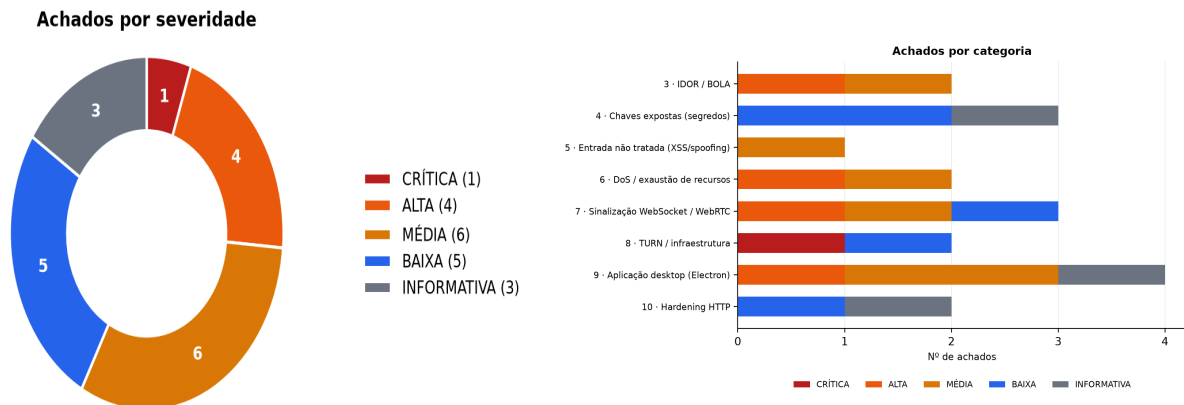
Escopo auditado: **crates/protocol**, **crates/signaling** (relay Axum/WebSocket + registry em memória + TURN), **apps/web** (Leptos SSR + hydrate/WASM), **desktop/** (Electron), e artefatos de implantação (**Dockerfile**, **docker-entrypoint.sh**, **fly.toml**, **.github/workflows**). Revisão de código estática, manual, arquivo por arquivo; histórico git varrido por segredos. Sem teste dinâmico contra ambiente vivo.

Nota metodológica. As cinco categorias do roteiro foram mapeadas para a stack: (1) *isolamento de inquilino* → não há banco/ORM; o mecanismo é o *registry* em memória chaveado por código de sala, com *room_code* + *peer_id* vinculados à conexão no servidor. (2) *Permissão no navegador* → o modelo de sala é plano (todo membro é igual, sem papel de admin/host), então quase não há operação privilegiada a verificar; a única checagem que importa (senha da sala) é feita no servidor. (3) *IDOR* → IDs de objeto são o código de sala (rota */api/rooms/:code* e *JoinRoom*) e os *peer_id* nas mensagens de sinalização. (4) *Segredos* → *Dockerfile*, *docker-entrypoint.sh*, *fly.toml*, CI, bundle do frontend e histórico git. (5) *Entrada não tratada* → no frontend, Leptos escapa por padrão e não há *inner_html/eval*; foco em CSS/atributos, *spoofing* por Unicode e injeção de comando no desktop.

1. Resumo executivo

A auditoria registrou **19 achados verificados** mais **14 pontos fortes** comprovados. Distribuição por severidade: **1** crítica, **4** alta, **6** média, **5** baixa, **3** informativa.

O risco central não está no vídeo P2P (protegido por DTLS-SRTP) e sim na **infraestrutura de relay TURN** (SSRF a metadados de nuvem e abuso de banda — F01), na **ausência de limites de recurso** no relay de sinalização (DoS remoto trivial e não autenticado — F02/F03) e na **cadeia de suprimento do app desktop** (auto-update sem verificação de assinatura — F04). Nenhum achado é uma violação direta de dados/RCE remota sem pré-condição — não há banco de dados nem funções de servidor além de `/ws` e `/api/rooms/:code`.



2. Pontos fortes (o que está protegido)

✓ Sem sink de XSS no frontend; escape automático do Leptos

`grep` por ``inner_html` / `set_inner_html` / `dangerously` / `eval` / `new Function`` em ``apps/web`` e ``desktop`` retorna vazio. O ``view!`` do Leptos escapa texto por padrão, então ``nick`/`room_name`/nomes de watchers/status` — todos controlados pelo cliente — renderizam com segurança (``apps/web/src/features/room/member_card.rs:249,278``).

✓ color nunca renderizado cru — allowlist de paleta

``components/palette.rs:21-27`` mapeia ``color`` por uma tabela fixa de 10 entradas e cai num ``#b0b8c1`` seguro para valor desconhecido; o binding ``style="--member: {...}"`` em ``member_card.rs:220`` não dá para injeção de CSS.

✓ peer_id gerado no servidor; from das mensagens é do servidor

``registry.rs:119,209`` usam ``Uuid::new_v4()``. Em ``ws.rs``, ``Offer`/`Answer`/`IceCandidate`` são retransmitidos com ``from: from.clone()`` (o ``peer_id`` da conexão), nunca um valor do cliente — impersonation por ID não é possível.

✓ Isolamento entre salas íntegro

Toda operação do registry resolve ``rooms.get(room_code)`` com o ``room_code`` vinculado à conexão no join (``registry.rs:relay/add_watcher/start_share/...``). Não há IDOR entre salas — um membro não consegue endereçar outra sala.

✓ Senha de sala: argon2id, verificada no servidor, sem bypass por vazio

``auth.rs`` usa o crate ``argon2`` (não implementação própria); ``registry.rs:check_optional_password`` exige senha não-vazia e ``verify_password`` correto; senha em branco só entra em sala explicitamente pública.

✓ Proteção contra brute force de senha existe (janela deslizante, por cliente)

``registry.rs:20-27,429-445``: ``MAX_PASSWORD_ATTEMPTS = 5`` em ``PASSWORD_ATTEMPT_WINDOW = 60 s``, escopo por cliente (não por sala, para não trancar terceiros); ``TooManyAttempts`` é devolvido mesmo se a senha desta vez estiver correta (ver F09 para a ressalva do proxy).

✓ Limite de membros aplicado no servidor

``registry.rs:205`` recusa join quando ``room.members.len() >= MAX_MEMBERS`` (10) — não confia na UI.

✓ Sem segredos no código, no histórico git ou no bundle do frontend

`git log -p --all` varrido — nenhum segredo commitado. `TURN\_SECRET` é um Fly secret (`fly.toml:12` documenta `fly secrets set`), não está em `fly.toml`. As credenciais TURN entregues ao browser são HMAC de curta duração cunhadas no servidor e só após autenticação (`turn.rs:58-75`, `media.rs`).

✓ coturn usa use-auth-secret (sem alocações anônimas)

`docker-entrypoint.sh:27-28` — `--use-auth-secret` + `--static-auth-secret`. (Hardening de peer-IP/cotas ainda falta: F01.)

✓ CI com privilégio mínimo e sem exposição de segredo a PR de fork

`.github/workflows/ci-cd.yml:37` `permissions: contents: read`; gatilho `pull\_request` (não `pull\_request\_target`); `secrets.FLY\_API\_TOKEN` só no job `deploy-web`, condicionado a `github.event\_name != 'pull\_request'`.

✓ Sem injeção de comando no desktop

Todos os `spawn`/`runCollectingStdout` usam array de argv, sem `shell: true` (`platform/run-command.ts`, `platform/linux/pipewire.ts`). As strings `binary`/`excludedBinaries` vindas do renderer são apenas comparadas (`===`, `.includes`), nunca passadas a um shell.

✓ Sem panic acionável por entrada hostil no servidor de sinalização

Cada `expect()` revisado: mutex do registry (`registry.rs:106`) nunca é mantido através de panic; relógio (`turn.rs:61`); HMAC aceita chave de qualquer tamanho (`turn.rs:66`); `ServerMessage` sempre serializa (`ws.rs:48`); argon2 sobre `&str` UTF-8 válido (`auth.rs:9`). JSON malformado no WS é silenciosamente ignorado (`ws.rs:64`).

✓ Seletor de captura explícito no desktop

`display-media.ts` implementa `setDisplayMediaRequestHandler` com uma janela de seleção própria — nunca captura tela cheia silenciosamente. Electron 43 (`contextIsolation`/`sandbox` ligados por padrão).

✓ Código de sala de fonte CSPRNG, não sequencial

`registry.rs:488-498`: 8 caracteres sobre um alfabeto de 31 símbolos sem ambiguidade, de `rand::rng()` (ChaCha) — $2^{39,6}$, não incremental, não previsível. (Ainda assim, ver F06 sobre o oráculo de enumeração.)

3. Pontos fracos (riscos centrais)

ID	Sev.	Risco
F01	CRÍTICA	coturn sem allowlist de peer-IP e sem cotas de banda/alocação
F02	ALTA	Servidor de sinalização sem limites (salas globais, conexões, tamanho/taxa de mensagem, canal ilimitado)
F03	ALTA	Reenvio de CreateRoom/JoinRoom no mesmo socket vaza salas e membros permanentemente
F04	ALTA	verifyUpdateCodeSignature: false desativa a verificação de assinatura no auto-update
F05	ALTA	Handshake /ws aceita qualquer Origin (sem allowlist)
F06	MÉDIA	GET /api/rooms/:code sem autenticação nem rate limit expõe nome/ocupação e serve de oráculo de enumeração
F07	MÉDIA	Sinalização Offer/Answer/IceCandidate/SetQuality aceita para qualquer co-membro sem relação de watch; cliente responde a Offer incondicionalmente
F08	MÉDIA	nick / room_name / color sem limite de tamanho nem sanitização de caracteres de controle
F09	MÉDIA	Limitador de brute force da senha confia em fly-client-ip sem validar proxy confiável
F10	MÉDIA	Renderer privilegiado carrega origem remota sem bloqueio de navegação/janelas
F11	MÉDIA	Pontes IPC potentes expostas à página remota sem checagem de origem/emissor

4. Achados detalhados por categoria

3 · IDOR / BOLA

ALTA

F03 — Reenvio de CreateRoom/JoinRoom no mesmo socket vaza salas e membros permanentemente

Arquivo:linha: crates/signaling/src/ws.rs:68-134; crates/signaling/src/registry.rs:380-409

```
ClientMessage::CreateRoom { .. } => {
    let (code, snapshot) = registry.create_room(.., tx.clone());
    room_code = Some(code);           // sobrescreve o anterior
    peer_id   = Some(snapshot.peer_id);
}
// ... ao fechar o socket:
if let (Some(room), Some(id)) = (room_code, peer_id) {
    registry.leave_room(&room, &id); // limpa SÓ a última sala/peer
}
```

Por que é explorável: Não há verificação de que a conexão já está vinculada a uma sala. Um segundo `CreateRoom` apenas sobrescreve as variáveis locais `room\_code`/`peer\_id`; a sala anterior mantém um `Member` com o `tx` deste socket, nunca fica vazia e `schedule\_empty\_room\_cleanup` nunca roda — a sala vaza para sempre. Variante: `JoinRoom` na mesma sala N vezes com `device\_id` distintos deixa N-1 membros órfãos que nunca são removidos na desconexão → a sala fica permanentemente 'cheia' e indeletável (negação direcionada a uma sala).

Impacto: Vazamento de memória ilimitado e não autenticado (OOM); negação de serviço direcionada a uma sala específica (trava em 10/10 para sempre).

Correção sugerida: Rejeitar `CreateRoom`/`JoinRoom` quando a conexão já tem `room\_code`/`peer\_id` (erro explícito), ou desfazer a associação anterior (`leave\_room`) antes de criar/entrar na nova. Garantir 1 `peer\_id` por conexão.

Critérios de aceite:

- Teste: 2º `CreateRoom` na mesma conexão retorna erro e NÃO cria uma segunda sala.
- Teste: N `JoinRoom` na mesma sala pelo mesmo socket => 1 membro; ao desconectar a sala esvazia e é agendada para remoção.
- Contagem de salas volta a zero após todos os sockets fecharem (teste com `test-util` e avanço de tempo além do grace period).

MÉDIA

F06 — GET /api/rooms/:code sem autenticação nem rate limit expõe nome/ocupação e serve de oráculo de enumeração

Arquivo:linha: crates/signaling/src/rooms_status.rs:7-24; apps/web/src/main.rs:41; crates/signaling/src/registry.rs:273-280; crates/signaling/src/registry.rs:488-498

```
pub async fn room_status_handler(
    State(registry): State<Registry>,
    Path(code): Path<String>,
) -> Json<RoomStatus> {
    match registry.room_status(&code) {
        Some(summary) => Json(RoomStatus { exists: true,
            name: Some(summary.name), member_count: Some(summary.member_count),
            requires_password: Some(summary.requires_password) }),
        None => Json(RoomStatus { exists: false, .. }),
    }
}
// ROOM_CODE_ALPHABET = 31 símbolos, ROOM_CODE_LENGTH = 8 => ~2^39,6
```

Por que é explorável: Referência direta ao objeto (o código) sem qualquer verificação de posse ou de membresia. Para qualquer código existente retorna o NOME escolhido por humano (potencialmente sensível: 'Reunião Diretoria — Fusão X'), a ocupação e se há senha. Sem rate limit, é um oráculo de existência que torna a enumeração observável; para uma sala pública, o código é a única credencial, então basta adivinhá-lo para depois entrar.

Impacto: Divulgação de informação (nome/atividade de salas) a qualquer um; reconhecimento e enumeração de salas; combinado com sala pública, acesso à sala.

Correção sugerida: Não devolver `name`/`member\_count` para chamadas não autenticadas — devolver apenas `{ exists, requires\_password }` (o mínimo para o fluxo de 'link morto'). Aplicar rate limit por IP neste endpoint. Opcional: código com mais entropia (>= 128 bits) para salas públicas, ou separar `room\_id` de um `join\_secret`.

Critérios de aceite:

- Resposta do endpoint não contém o nome da sala nem a contagem de membros.

- Rate limit por IP aplicado (teste: N+1 chamadas em T segundos => 429).
- Fluxo de 'sala não encontrada' na UI continua funcionando.

4 · Chaves expostas (segredos)

BAIXA

F13 — Realm com default público e ausência de validação de startup rejeitando TURN_SECRET fraco/placeholder

Arquivo:linha: docker-entrypoint.sh:11-37; crates/signaling/src/turn.rs:31-50

```
if [ -n "$TURN_SECRET" ] && [ -n "$TURN_EXTERNAL_IP" ]; then
    turnserver ... --realm="$${TURN_REALM:-screenshot}" \
        --static-auth-secret="$TURN_SECRET" ...
# from_vars: só rejeita string vazia; qualquer valor não-vazio serve
```

Por que é explorável: `--realm` cai num default público (`screenshot`) sem asserção de startup. Mais relevante: `TurnConfig::from\_vars` só recusa `TURN\_SECRET` vazio — um valor placeholder fraco (ex.: `changeme`) é aceito silenciosamente por coturn e por `mint\_credentials`, sem nenhum gate que rejeite defaults conhecidos.

Impacto: Segredo TURN fraco não detectado no boot; realm previsível.

Correção sugerida: Validar no startup: `TURN\_SECRET` com comprimento/entropia mínimos e não pertencente a uma denylist de placeholders; abortar o processo se falhar. Tornar `TURN\_REALM` obrigatório quando TURN está ligado (sem default).

Critérios de aceite:

- Processo recusa iniciar com `TURN\_SECRET` curto ou em denylist (teste).
- Sem `TURN\_REALM` explícito com TURN ligado => erro de configuração no boot.

BAIXA

F14 — Senha da sala guardada em texto puro em sessionStorage

Arquivo:linha: apps/web/src/infra/storage.rs:32-64

```
pub struct RoomSession {
    pub nick: String,
    pub color: String,
    pub password: Option<String>, // texto puro, chave screen_share_room_session_<code>
}
```

Por que é explorável: A senha da sala é persistida em `sessionStorage` (escopo de aba, apagada ao fechar) para permitir o rejoin silencioso após reload. É uma escolha deliberada, mas qualquer script na origem (um XSS, ou a superfície das pontes desktop) consegue lê-la e exfiltrá-la junto com o nick.

Impacto: Exposição da senha da sala a script na origem (defesa em profundidade).

Correção sugerida: Preferir não persistir a senha: manter um token de rejoin de curta duração emitido pelo servidor em vez da senha. Se mantida, documentar o risco e reforçar CSP (F12).

Critérios de aceite:

- Rejoin após reload funciona sem gravar a senha em claro (token de sessão), OU risco aceito e documentado em ADR com CSP aplicada.

INFORMATIVA

F19 — E-mail pessoal do mantenedor versionado em desktop/package.json

Arquivo:linha: desktop/package.json:5-8

```
"author": { "name": "hiroshimorowaka", "email": "guilhermecabral1204@gmail.com" }
```

Por que é explorável: Não é segredo, mas é PII exposta e reutilizável para phishing/enumeração.

Impacto: Exposição de PII.

Correção sugerida: Usar um e-mail de projeto/alias em vez de e-mail pessoal.

Critérios de aceite:

- `author.email` aponta para um endereço de projeto.

5 · Entrada não tratada (XSS/spoofing)

MÉDIA

F08 — nick / room_name / color sem limite de tamanho nem sanitização de caracteres de controle

Arquivo:linha: crates/protocol/src/client.rs:9-23; apps/web/src/features/home/create_room.rs:43-59; crates/signaling/src/registry.rs:109-160; apps/web/src/features/room/member_card.rs:249; apps/web/src/features/room/member_card.rs:278

```
CreateRoom { nick: String, password: Option<String>, room_name: String,
              color: String, device_id: String }
// create_room.rs: só `.trim()` + não-vazio.
// registry.rs: nick/color/room_name armazenados verbatim e enviados a todos
// via PeerJoined / snapshot Joined.
// member_card.rs: {m.nick} é renderizado (Leptos escapa HTML – sem XSS),
// mas sem limite de tamanho e sem filtrar \n, RTL override, homoglifos.
```

Por que é explorável: Não é XSS: o `view!` do Leptos escapa texto por padrão (ver Pontos Fortes). Mas: (a) um `nick` de vários MB é guardado em memória e RETRANSMITIDO a cada membro em `PeerJoined` e repetido no snapshot de entrada (amplificação) e injetado no DOM de todos; (b) `room\_name` volta sem autenticação em `/api/rooms/:code` (F06); (c) caracteres de controle / override RTL / homoglifos em `nick` renderizam direto na UI dos outros membros, permitindo passar-se por outro membro ou pela etiqueta 'você'.

Impacto: Amplificação de broadcast (DoS); falsificação de identidade visual na sala.

Correção sugerida: Validar no servidor (fonte da verdade) e espelhar no cliente: `MAX\_NICK\_LEN` (~32), `MAX\_ROOM\_NAME\_LEN` (~64), `color` restrito à allowlist da paleta (rejeitar, não só cair no default). Remover/rejeitar caracteres de controle e de formatação bidirecional. Normalizar Unicode (NFC).

Critérios de aceite:

- `CreateRoom`/`JoinRoom` com `nick` acima do limite é rejeitado com erro dedicado.
- `color` fora da paleta é rejeitado no servidor.
- `nick` com `\n`/`\u202E` é recusado ou saneado; teste unitário cobre.

6 · DoS / exaustão de recursos

ALTA

F02 — Servidor de sinalização sem limites (salas globais, conexões, tamanho/taxa de mensagem, canal ilimitado)

Arquivo:linha: crates/signaling/src/ws.rs:36-66; crates/signaling/src/registry.rs:90-160; apps/web/src/main.rs:39-55

```
let (tx, mut rx) = mpsc::unbounded_channel::<ServerMessage>(); // ws.rs:43
while let Some(Ok(msg)) = ws_receiver.next().await {           // ws.rs:62
    let Message::Text(text) = msg else { continue };
    let Ok(client_msg) = serde_json::from_str::<ClientMessage>(&text) else { continue };
    // sem limite de tamanho, sem rate limit, sem timeout de inatividade
    // registry.rs: HashMap<String, Room> sem MAX_ROOMS nem limite por IP
```

Por que é explorável: Nenhum limite global de salas, de conexões WebSocket simultâneas, de mensagens por segundo, nem `idle` timeout/heartbeat. O canal de saída é `mpsc::unbounded\_channel`: uma `sdp`/`candidate` de dezenas de MB (limite default do axum-ws ~64 MiB) repassada a um receptor lento acumula sem teto na fila dele. Um único socket sem autenticação prévia pode criar salas, abrir milhares de conexões ociosas (slowloris) ou inundar sinalização. A VM de produção tem 256 MB e `min\_machines\_running` = 0.

Impacto: Negação de serviço remota, não autenticada e trivial (esgotamento de memória / descritores) derrubando todo o serviço.

Correção sugerida: Definir consts nomeadas: `MAX\_ROOMS`, `MAX\_ROOMS\_PER\_CLIENT`, `MAX\_WS\_CONNECTIONS`, `MAX\_MESSAGE\_BYTES`, `MAX\_MSGS\_PER\_SEC`. Trocar por `mpsc::channel(N)` com backpressure. Aplicar `WebSocketUpgrade::max\_message\_size`. Adicionar heartbeat ping/pong com timeout de inatividade. Rejeitar `CreateRoom` acima do teto por IP (`fly-client-ip`).

Critérios de aceite:

- Teste de integração: 10 000 `CreateRoom` de um cliente resultam em <= `MAX\_ROOMS\_PER\_CLIENT` salas e erro explícito depois.
- Mensagem de texto acima de `MAX\_MESSAGE\_BYTES` fecha a conexão sem alocar o corpo inteiro.
- Conexão sem tráfego é encerrada após o timeout de inatividade (teste com `tokio::time`).
- Canal de saída é limitado; envio a receptor saturado aplica backpressure em vez de crescer.

MÉDIA

F09 — Limitador de brute force da senha confia em fly-client-ip sem validar proxy confiável

Arquivo:linha: crates/signaling/src/ws.rs:12-24; crates/signaling/src/registry.rs:20-27; crates/signaling/src/registry.rs:429-445

```
fn client_key(headers: &HeaderMap) -> String {
    headers.get("fly-client-ip")
```

```

        .and_then(|v| v.to_str().ok())
        .map(str::to_owned)
        .unwrap_or_else(|| "unknown".to_string())
    }

```

Por que é explorável: O valor do header é usado como chave do balde de tentativas de senha (`MAX\_PASSWORD\_ATTEMPTS = 5 / 60 s`). Na Fly o header é reescrito pela borda e é confiável; mas não há verificação de que a requisição veio por um proxy confiável. Em qualquer implantação fora da Fly (ou se o app ficar acessível diretamente), o atacante rotaciona `fly-client-ip` a cada conexão e ignora totalmente o limite → brute force ilimitado da senha da sala. Quando o header está ausente, todos caem no balde único `unknown`, então 5 falhas de um atacante trancam todos os usuários daquela sala (DoS).

Impacto: Brute force ilimitado de senha de sala (fora da Fly / acesso direto); ou negação de entrada para usuários legítimos (balde global).

Correção sugerida: Só confiar em `fly-client-ip`/`X-Forwarded-For` quando a conexão vier de uma lista de proxies confiáveis (env `TRUSTED\_PROXIES`); caso contrário usar o IP real da conexão TCP (`ConnectInfo`). Nunca cair em uma chave constante compartilhada — usar o IP de peer real como fallback.

Critérios de aceite:

- Teste: variar `fly-client-ip` a cada request sem vir de proxy confiável NÃO aumenta o número de tentativas permitidas.
- Sem header e sem proxy confiável, a chave é o IP de peer real, não `unknown`.
- Config de proxies confiáveis documentada.

7 · Sinalização WebSocket / WebRTC

ALTA

F05 — Handshake /ws aceita qualquer Origin (sem allowlist)

Arquivo:linha: crates/signaling/src/ws.rs:26-34

```

pub async fn ws_handler(
    State(registry): State<Registry>,
    State(turn): State<Option<TurnConfig>>,
    headers: HeaderMap,
    ws: WebSocketUpgrade,
) -> impl IntoResponse {
    let client_key = client_key(&headers);
    ws.on_upgrade(move |socket| handle_socket(socket, registry, turn, client_key))
    // nenhuma checagem de header Origin
}

```

Por que é explorável: Qualquer página web (`evil.com`) pode abrir `wss://.../ws` e falar todo o protocolo de sinalização. Não há cookies de sessão, então não é um Cross-Site WebSocket Hijacking clássico de roubo de sessão; mas qualquer origem passa a poder criar salas, entrar em salas públicas, cunhar credenciais TURN e conduzir os abusos de F02/F03/F01. O OWASP WebSocket Cheat Sheet recomenda explicitamente uma allowlist de `Origin` no handshake.

Impacto: Amplia a superfície de F01/F02/F03 para qualquer site que a vítima visite; abuso de recursos e de TURN a partir de origens arbitrárias.

Correção sugerida: Validar `Origin` contra uma allowlist configurável (a própria origem do app + `SCREEN\_SHARE\_URL` do desktop, se aplicável) e recusar o upgrade (403) caso não bata. Manter isso como defesa em profundidade, não como autenticação.

Critérios de aceite:

- `Origin: https://evil.com` no handshake => resposta 403, sem `101 Switching Protocols`.
- `Origin` da própria app => upgrade normal; teste e2e continua verde.
- Allowlist vem de configuração/env, com um `const` documentando o default.

MÉDIA

F07 — Sinalização Offer/Answer/IceCandidate/SetQuality aceita para qualquer co-membro sem relação de watch; cliente responde a Offer incondicionalmente

Arquivo:linha: crates/signaling/src/ws.rs:163-219; apps/web/src/session/handler.rs:258-351

```

ClientMessage::Offer { to, sdp } => {
    if let (Some(room), Some(from)) = (&room_code, &peer_id) {
        registry.relay(room, &to, ServerMessage::Offer { from: from.clone(), sdp });
    }
}
// handler.rs: ServerMessage::Offer { from, sdp } => cria RTCPeerConnection,
// seta ontrack -> <video id="video-{from}">, create_answer, envia Answer,
// troca ICE — sem verificar se existe relação de watch com `from`.

```


Por que é explorável: O servidor isola por sala (só entrega a membros da MESMA sala e `from` é definido pelo servidor — bom), mas não exige que os dois lados tenham concordado em se conectar. Um membro malicioso pode enviar `Offer { to: vítima }`; o cliente da vítima cria uma `RTCPeerConnection`, troca ICE (revelando candidatos host/srflx — IP de LAN e público — ao atacante sem consentimento) e responde. Também permite spam de renegociação, injeção de candidatos e `QualityRequested` para degradar o stream de outro espectador.

Impacto: Divulgação de endereço IP (LAN + público) entre co-membros sem consentimento; consumo forçado de recursos; perturbação de conexões P2P dentro da sala.

Correção sugerida: No servidor: só relayar `Offer`/`Answer`/`IceCandidate`/`SetQuality` quando existir uma relação (sharer, viewer) registrada em `watchers` entre `from` e `to`. No cliente: ignorar `Offer` de um `from` que o usuário não escolheu assistir e que não é um espectador registrado.

Critérios de aceite:

- Teste: `Offer` de A para B sem B ter pedido `WatchShare` de A (nem A de B) é descartado pelo servidor.
- Teste de cliente: `ServerMessage::Offer` de peer não relacionado não cria `RTCPeerConnection`.
- Cenário de watch legítimo (2 abas) continua estabelecendo mídia.

BAIXA

F15 — add_watcher aceita sharer_id arbitrário; ReportLatency aceita ping arbitrário

Arquivo:linha: crates/signaling/src/registry.rs:311-327; crates/signaling/src/registry.rs:349-366

```
pub fn add_watcher(&self, room_code: &str, sharer_id: &str, viewer_id: &str) {
    room.watchers.entry(sharer_id.to_string()).or_default().insert(viewer_id.to_string());
    // sharer_id não é validado como membro nem como sharer ativo
}
// report_latency: `ms: u32` do cliente é retransmitido como o ping daquele peer
```

Por que é explorável: `add\_watcher` cria entradas em `watchers` para `sharer\_id` que não é membro nem está compartilhando (poluição de estado, `WatchersChanged` espúrio). `ReportLatency` aceita qualquer `ms` e o servidor retransmite como o ping medido daquele peer (spoofing cosmético do indicador de ping alheio).

Impacto: Poluição de estado do mapa de watchers; falsificação do indicador de ping de outros membros.

Correção sugerida: `add\_watcher`: ignorar se `sharer\_id` não estiver em `room.members` (ou não em `room.sharers`). `report\_latency`: só aceitar para o próprio `peer\_id` da conexão (já é o caso) e limitar `ms` a um teto plausível.

Critérios de aceite:

- `WatchShare` para um `sharer\_id` inexistente não cria entrada em `watchers`.
- `ReportLatency` acima do teto é descartado.

8 · TURN / infraestrutura

CRÍTICA

F01 — coturn sem allowlist de peer-IP e sem cotas de banda/alocação

Arquivo:linha: docker-entrypoint.sh:24-37; fly.toml:23-25; crates/signaling/src/turn.rs:16

```
turnserver \
  --no-cli --fingerprint --use-auth-secret \
  --static-auth-secret="$TURN_SECRET" \
  --realm="{TURN_REALM:-screenshare}" \
  --listening-port=3478 \
  --min-port=... --max-port=... \
  --external-ip="$TURN_EXTERNAL_IP" \
  --log-file=stdout --no-tls --no-dtls &
# sem --denied-peer-ip / --allowed-peer-ip
# sem --total-quota / --user-quota / --max-bps / --bps-capacity
```

Por que é explorável: O relay exige credencial (`--use-auth-secret`), mas qualquer participante de qualquer sala recebe uma credencial TURN válida por 6 h no snapshot `Joined` (`ServerMessage::Joined.turn`). Para uma sala pública, isso é qualquer pessoa que conheça ou adivinhe o código. De posse da credencial, um atacante pode pedir alocações e mandar o coturn encaminhar pacotes para endereços arbitrários — inclusive `169.254.169.254` (metadados de nuvem), faixas RFC1918/`fdaa::16` (rede privada 6PN da organização na Fly) e `127.0.0.0/8`. coturn recente nega loopback por padrão, mas NÃO nega link-local nem RFC1918. Sem `--total-quota`/`--max-bps` o mesmo relay serve de amplificador de tráfego às custas da banda/conta da Fly.

Impacto: SSRF a partir da infraestrutura de relay: varredura e acesso a serviços internos da Fly e ao endpoint de metadados (potencial roubo de tokens do ambiente). Abuso de banda / negação de serviço por exaustão de alocações.

Correção sugerida: Adicionar `--denied-peer-ip=0.0.0.0-0.255.255.255`, `10.0.0.0-10.255.255.255`, `100.64.0.0-100.127.255.255`, `169.254.0.0-169.254.255.255`, `172.16.0.0-172.31.255.255`, `192.168.0.0-192.168.255.255`, `::1`, `fc00::7`, `fe80::10` (e liberar

só o necessário com `--allowed-peer-ip`). Definir `--total-quota`, `--user-quota`, `--max-bps`, `--bps-capacity`, `--no-multicast-peers`. Reduzir `CREDENTIAL\_TTL` para ~1 h.

Critérios de aceite:

- `turnutils\_peer` / `turnutils\_uclient` contra 169.254.169.254 e um IP RFC1918 falham (peer negado).
- Alocações acima da cota são rejeitadas; `--max-bps` visível em `turnserver -h` do processo em execução.
- Teste de fumaça de sala real (2 abas, mídia P2P) continua passando.
- `CREDENTIAL\_TTL` documentado com o novo valor e razão.

BAIXA

F16 — --no-tls --no-dtls: canal de controle STUN/TURN em texto puro

Arquivo:linha: docker-entripoint.sh:35-37; fly.toml:25
turnserver ... --no-tls --no-dtls &
TURN_URLS = "turn:137.66.9.162:3478" # não "turns:"

Por que é explorável: O canal de controle STUN/TURN trafega sem TLS/DTLS. A mídia continua protegida por SRTP (a confidencialidade da tela não é perdida), mas some uma camada e o relay não atravessa firewalls restritivos que só liberam 443/TLS.

Impacto: Metadados de sinalização TURN observáveis na rede; menor alcance de conectividade.

Correção sugerida: Habilitar `turns:` (TLS) na 5349 com certificado; manter `turn:` como fallback.

Critérios de aceite:

- coturn escuta `turns:` com certificado válido; `TURN\_URLS` inclui a URL `turns:`.
- Teste de sala real com TURN forçado (`iceTransportPolicy: relay`) passa via TLS.

9 · Aplicação desktop (Electron)

ALTA

F04 — verifyUpdateCodeSignature: false desativa a verificação de assinatura no auto-update

Arquivo:linha: desktop/package.json:80-87; desktop/src/main/updates.ts:32-44
"win": {
 "target": ["nsis", "portable"],
 "icon": "icons/app-icon.png",
 "verifyUpdateCodeSignature": false
}
// updates.ts: autoUpdater.checkForUpdatesAndNotify() – instala ao sair

Por que é explorável: Com `verifyUpdateCodeSignature: false`, o `electron-updater` (NSIS) não confere que o novo instalador foi assinado pelo mesmo publicador. A confiança no update passa a ser apenas 'veio via HTTPS do nosso GitHub Releases'. Comprometimento do token/conta do GitHub, de um asset de release ou do pipeline resulta em execução de código persistente e silenciosa em todos os clientes Windows ('instalar ao sair'). `asar: false` (linha 52) ainda distribui o app sem integridade de bundle.

Impacto: RCE persistente e silenciosa em máquinas de usuários finais (cadeia de suprimento do desktop).

Correção sugerida: Assinar os builds Windows e remover `verifyUpdateCodeSignature: false` (ou defini-lo como `true` explicitamente). Habilitar `asar: true`. Considerar `electron-updater` com chave pública própria e verificação de hash/assinatura do `latest.yml`.

Critérios de aceite:

- Build de release Windows é assinado; `verifyUpdateCodeSignature` não aparece como `false`.
- Um `latest.yml`/instalador adulterado é rejeitado pelo updater (teste manual documentado).
- `asar` habilitado; `files` do electron-builder revisado.

MÉDIA

F10 — Renderer privilegiado carrega origem remota sem bloqueio de navegação/janelas

Arquivo:linha: desktop/src/main/window.ts:16-36; desktop/src/features/screen-share/picker.ts:35-48
mainWindow = new BrowserWindow({
 width: 1100, height: 750, show: false,
 webPreferences: { preload: path.join(__dirname, '..', 'preload.js') },
});
mainWindow.loadURL(APP_URL); // https://screen-share-h0rb5w.fly.dev/
// sem setWindowOpenHandler, sem will-navigate / will-redirect,
// sem contextIsolation/sandbox/webSecurity explícitos

Por que é explorável: A janela principal carrega uma origem REMOTA e não há `webContents.setWindowOpenHandler`, nem handler de `will-navigate`/`will-redirect`. Um XSS ou open-redirect nessa origem (ou sequestro dela) pode navegar o renderer privilegiado para conteúdo do atacante, que então tem acesso às pontes IPC expostas (`desktopShare`, `picker`, `desktopAudio` — ver F11). Os defaults do Electron 43 (`contextIsolation`/`sandbox` ligados) evitam acesso direto ao Node, mas não as pontes. As flags de segurança não são fixadas explicitamente (fragilidade).

Impacto: Escalada de um XSS/redirect na web para o contexto do app desktop, com acesso às pontes IPC (ver impacto em F11).

Correção sugerida: `webContents.setWindowOpenHandler(() => ({ action: 'deny' }));` bloquear `will-navigate`/`will-redirect` para fora da origem do app; fixar `contextIsolation: true`, `sandbox: true`, `nodeIntegration: false`, `webSecurity: true` explicitamente; considerar `session` com CSP injetada.

Critérios de aceite:

- `window.open(...)` e navegação para `https://evil.com` a partir do renderer são bloqueadas (teste `_electron`).
- `webPreferences` fixa as 4 flags explicitamente.
- e2e desktop continua verde.

MÉDIA

F11 — Pontes IPC potentes expostas à página remota sem checagem de origem/emissor

Arquivo:linha: `desktop/src/preload.ts:9-49`; `desktop/src/features/audio-share/ipc.ts:12-26`; `desktop/src/platform/windows/audio.ts:19-32`; `desktop/src/features/screen-share/quick-share.ts:11-32`

```
contextBridge.exposeInMainWorld('desktopAudio', {
  start: (target) => ipcRenderer.invoke('start-audio-loopback', target),
  stop: () => ipcRenderer.invoke('stop-audio-loopback'),
});
contextBridge.exposeInMainWorld('picker', { listAudioApps: () =>
  ipcRenderer.invoke('list-audio-apps'), .. });
contextBridge.exposeInMainWorld('desktopShare', { linkReady: (l) =>
  ipcRenderer.send('desktop-share:link-ready', l), .. });
// ipc.ts: ipcMain.handle('start-audio-loopback', (_e, target) => startAudioLoopback(target)) — sem
  checar _e.senderFrame
```

Por que é explorável: O preload é compartilhado com a janela principal, que carrega a origem remota. Nenhum `ipcMain.on/handle` verifica `event.senderFrame`/origem. A partir de um XSS na web (ou via F10) a página pode: `picker.listAudioApps()` / `desktopAudio.start({mode:'screen',excludedBinaries:[...]})` → enumerar apps em execução e nomes/caminhos de executáveis, e INICIAR uma captura real de áudio do sistema sem consentimento nem seletor; no Windows o PCM misturado é então empurrado para o renderer via `desktop-audio-pcm-chunk` (`platform/windows/audio.ts:26-29`) — captura encoberta do áudio do sistema exfiltrável pela página. `desktopShare.linkReady("qualquer coisa")` sobrescreve a área de transferência; `memberJoined` gera notificação de SO com texto arbitrário.

Impacto: Captura encoberta de áudio do sistema (Windows); enumeração de processos do usuário; sequestro da área de transferência; spoofing de notificação.

Correção sugerida: Verificar `event.senderFrame` (origem == origem do app) em todo handler `ipcMain`. Não empurrar PCM para o renderer sem um sinal de sessão ativa iniciada pelo próprio app. Restringir `linkReady` a um formato de link esperado. Idealmente expor as pontes só quando o conteúdo local/confiável estiver carregado.

Critérios de aceite:

- Todo handler IPC rejeita mensagens cujo `senderFrame` não seja a origem do app (teste unitário).
- `desktopAudio.start` sem uma sessão de compartilhamento ativa não emite `desktop-audio-pcm-chunk`.
- `linkReady` valida o formato antes de escrever no clipboard.

INFORMATIVA

F18 — asar: false — app empacotado sem integridade de bundle

Arquivo:linha: `desktop/package.json:52`
`"asar": false`

Por que é explorável: Sem `asar` (e sem `asintegrity`), os arquivos do app ficam soltos no disco, facilitando adulteração local persistente do código do app.

Impacto: Menor resistência a adulteração local; sem verificação de integridade do bundle.

Correção sugerida: `asar: true` e, no Electron recente, habilitar `asarIntegrity`.

Critérios de aceite:

- Build empacotado usa `asar`; verificação de integridade habilitada.

10 · Hardening HTTP

BAIXA

F12 — Sem Content-Security-Policy / HSTS / X-Content-Type-Options / Permissions-Policy

Arquivo:linha: apps/web/src/main.rs:39-56; apps/web/src/app.rs:14-39

```
let app = Router::new()
    .leptos_routes(&leptos_options, routes, move || shell(..))
    .fallback(leptos_axum::file_and_error_handler(shell))
    .with_state(leptos_options)
    .merge(signaling_router);
// nenhum middleware tower-http de cabeçalhos de segurança
```

Por que é explorável: Nenhum middleware adiciona cabeçalhos de segurança. Sem CSP, um XSS hipotético (ou a superfície de F10/F11) roda sem restrição; sem `Strict-Transport-Security` o `force\_https` da Fly só redireciona; sem `Permissions-Policy` não há restrição declarada de `display-capture`/`camera`/`microphone`; sem `X-Content-Type-Options: nosniff` sobra sniffing de MIME nos assets.

Impacto: Defesa em profundidade ausente; menor contenção de XSS e de abuso das pontes desktop.

Correção sugerida: Adicionar `tower\_http::set\_header`/`SetResponseHeaderLayer` (ou `tower-helmet`) com CSP restritiva (`default-src 'self';` `connect-src 'self' wss://... stun:/turn:`; `img-src 'self' data:`; `style-src 'self' fonts.googleapis.com`; `font-src fonts.gstatic.com`), `HSTS`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, `Permissions-Policy` restrito.

Critérios de aceite:

- Resposta HTTP inclui CSP, HSTS, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`.
- App e fontes do Google carregam sem violação de CSP no console.
- Teste de fumaça (criar/entrar/assistir) continua passando.

INFORMATIVA

F17 — Ação de GitHub sem pin de versão (setup-flyctl@master)

Arquivo:linha: .github/workflows/ci-cd.yml (job deploy-web)

```
- uses: superfly/flyctl-actions/setup-flyctl@master
```

Por que é explorável: `@master` acompanha o branch: um comprometimento do repositório da ação executa código arbitrário no runner de deploy, que tem acesso a `secrets.FLY\_API\_TOKEN`.

Impacto: Risco de cadeia de suprimento no pipeline de deploy.

Correção sugerida: Fixar a ação por SHA de commit (ou tag imutável).

Critérios de aceite:

- Todas as `uses:` de terceiros fixadas por SHA.

5. Recomendações priorizadas

Prio.	Ação
P1	Endurecer o coturn (F01): `--denied-peer-ip` para loopback/link-local/RFC1918/6PN, `--total-quota`/`--user-quota`/`--max-bps`/`--bps-capacity`, `--no-multicast-peers`; reduzir o TTL da credencial. Bloqueia SSRF a metadados de nuvem e abuso de banda.
P1	Impor limites de recurso no servidor de sinalização (F02, F03): `MAX_ROOMS` global e por IP, teto de conexões, `max_message_size`, rate limit de mensagens, heartbeat + idle timeout, canal mpvc limitado; rejeitar `CreateRoom`/`JoinRoom` em conexão já vinculada.
P1	Restaurar a integridade do auto-update do desktop (F04): assinar os builds Windows, remover `verifyUpdateCodeSignature: false`, habilitar `asar`.
P2	Allowlist de `Origin` no handshake `/ws` (F05) e validação de proxy confiável para o `client_key` do limitador de senha (F09).
P2	Autorização de sinalização par-a-par (F07): só relayar/aceitar `Offer`/`Answer`/`IceCandidate`/`SetQuality` quando existir relação (sharer, viewer) registrada; cliente ignora `Offer` não solicitada.
P2	Minimizar `GET /api/rooms/:code` (F06): não devolver nome/ocupação sem autenticação; rate limit por IP.
P2	Desktop: bloquear navegação/`window.open` do renderer e verificar `event.senderFrame` em todo handler IPC; não empurrar PCM sem sessão ativa (F10, F11).
P3	Validação de entrada no protocolo (F08, F15): limites de tamanho para `nick`/`room_name`, `color` restrito à paleta, rejeição de caracteres de controle/bidi, teto para `ReportLatency`, `add_watcher` só para membros.
P3	Cabeçalhos de segurança / CSP no app Axum (F12); validação de startup do `TURN_SECRET` e do realm (F13); `turns:` TLS no coturn (F16).
P3	Higiene: token de rejoin em vez de senha em `sessionStorage` (F14); pin de ações de CI por SHA (F17); e-mail de projeto no `package.json` (F19).

6. Issues para o GitHub

Cada bloco abaixo é o texto completo de uma issue em Markdown, pronto para copiar e colar. Achados triviais relacionados foram agrupados numa issue única.

--- ISSUE 1 ---

Título: [Segurança] coturn sem allowlist de peer-IP e sem cotas (SSRF a metadados + abuso de banda)

Labels sugeridas: security, severity:critical, infra

Problema

O `docker-entrypoint.sh` sobe o coturn com `--use-auth-secret` (bom), mas **sem** `--denied-peer-ip` e **sem** cotas de banda/alocação. Qualquer participante de qualquer sala recebe uma credencial TURN válida por 6 h no snapshot `Joined`; para uma sala pública, isso é qualquer pessoa que conheça o código.

Por que é explorável

Com a credencial, um atacante pede alocações e faz o coturn encaminhar pacotes para endereços arbitrários: `169.254.169.254` (metadados de nuvem), RFC1918, `fd:::16` (6PN da Fly), etc. coturn recente nega loopback por padrão, mas **não** nega link-local nem RFC1918. Sem `--total-quota`/`--max-bps`, o relay também vira amplificador de tráfego.

Evidência

```
`docker-entrypoint.sh:24-37`
```sh
turnserver --no-cli --fingerprint --use-auth-secret \
 --static-auth-secret="$TURN_SECRET" --realm="$TURN_REALM:-screenshare" \
 --listening-port=3478 --min-port=... --max-port=... \
 --external-ip="$TURN_EXTERNAL_IP" --log-file=stdout --no-tls --no-dtls &
sem --denied-peer-ip / --allowed-peer-ip
sem --total-quota / --user-quota / --max-bps / --bps-capacity
```

`crates/signaling/src/turn.rs:16` — `CREDENTIAL_TTL = 6 h`.
```

Impacto

SSRF a partir da infraestrutura de relay (varredura interna + roubo potencial de tokens do endpoint de metadados); abuso de banda / DoS por exaustão de alocações.

Sugestão de correção

- `--denied-peer-ip` para `0.0.0.0/8`, `10/8`, `100.64/10`, `169.254/16`, `172.16/12`, `192.168/16`, `:::1`, `fc00::7`, `fe80::/10`; liberar só o necessário com `--allowed-peer-ip`.
- `--total-quota`, `--user-quota`, `--max-bps`, `--bps-capacity`, `--no-multicast-peers`.
- Reduzir `CREDENTIAL\_TTL` para ~1 h.

Critérios de aceite

- [] `turnutils\_uclient` contra `169.254.169.254` e um IP RFC1918 falham (peer negado).
- [] Alocações acima da cota são rejeitadas; `--max-bps` presente no processo em execução.
- [] Teste de sala real (2 abas, mídia P2P) continua passando.
- [] `CREDENTIAL\_TTL` atualizado e comentado com a razão.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Servidor de sinalização sem limites de recurso (salas, conexões, tamanho/taxa de mensagem, canal ilimitado)

Labels sugeridas: security, severity:high, dos

Problema

Não há `MAX\_ROOMS` global nem por IP, teto de conexões WebSocket, limite de tamanho/taxa de mensagem, `idle timeout` ou heartbeat. O canal de saída é `mpsc::unbounded\_channel`.

Por que é explorável

Um único socket (sem autenticação prévia) pode criar salas em massa, abrir milhares de conexões ociosas (slowloris) ou inundar sinalização. Uma `sdp` de dezenas de MB (limite default do axum-ws ~64 MiB) repassada a um receptor lento cresce sem teto na fila dele. A VM de produção tem 256 MB e `min\_machines\_running = 0`.

Evidência

```
`crates/signaling/src/ws.rs:43,62-66`
```rust
let (tx, mut rx) = mpsc::unbounded_channel::(<ServerMessage>());
while let Some(Ok(msg)) = ws_receiver.next().await {
 let Message::Text(text) = msg else { continue };
 let Ok(client_msg) = serde_json::from_str::(<ClientMessage>(&text)) else { continue };
 // sem limite de tamanho, sem rate limit, sem timeout
}

`crates/signaling/src/registry.rs:90-160` — `HashMap<String, Room>` sem teto.
```

## Impacto

Negação de serviço remota, não autenticada e trivial (OOM / esgotamento de descritores).

```
Sugestão de correção
`const` nomeadas: `MAX_ROOMS`, `MAX_ROOMS_PER_CLIENT`, `MAX_WS_CONNECTIONS`, `MAX_MESSAGE_BYTES`,
`MAX_MSGS_PER_SEC`. `mpsc::channel(N)` com backpressure. `WebSocketUpgrade::max_message_size`. Heartbeat
ping/pong + idle timeout. Rate limit de `CreateRoom` por `fly-client-ip`.
```

```
Critérios de aceite
- [] 10 000 `CreateRoom` de um cliente => <= `MAX_ROOMS_PER_CLIENT` e erro explícito depois.
- [] Texto acima de `MAX_MESSAGE_BYTES` fecha a conexão sem alocar o corpo inteiro.
- [] Conexão ociosa é encerrada após o timeout (teste com `tokio::time`).
- [] Canal de saída limitado; receptor saturado sofre backpressure.
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Reenvio de CreateRoom/JoinRoom no mesmo socket vaza salas e membros permanentemente

Labels sugeridas: security, severity:high, dos

## Problema

A conexão não verifica se já está vinculada a uma sala. Um segundo `CreateRoom` sobrescreve `room\_code`/`peer\_id` locais; a sala anterior mantém um `Member` com o `tx` do socket, nunca esvazia e nunca é removida.

## Por que é explorável

Loop de `CreateRoom` num socket => salas permanentes ilimitadas (OOM). Variante: `JoinRoom` na mesma sala N vezes com `device\_id` distintos => N-1 membros órfãos nunca removidos => sala presa em 10/10 e indeletável (negação direcionada).

## Evidência

```
`crates/signaling/src/ws.rs:68-134`
```rust
ClientMessage::CreateRoom { .. } => {
    let (code, snapshot) = registry.create_room(.., tx.clone());
    room_code = Some(code);           // sobrescreve
    peer_id = Some(snapshot.peer_id);
}
// ao fechar: registry.leave_room(&room, &id) -> limpa SÓ a última
```
`crates/signaling/src/registry.rs:380-409` – cleanup só quando `members.is_empty()`.
```

## Impacto

Vazamento de memória não autenticado (OOM); DoS direcionado a uma sala.

## Sugestão de correção

Rejeitar `CreateRoom`/`JoinRoom` quando a conexão já tem `room\_code`/`peer\_id`, ou desfazer a associação anterior antes. Garantir 1 `peer\_id` por conexão.

## Critérios de aceite

```
- [] 2º `CreateRoom` na mesma conexão retorna erro e não cria segunda sala.
- [] N `JoinRoom` na mesma sala pelo mesmo socket => 1 membro; desconectar esvazia a sala.
- [] Contagem de salas volta a zero após todos os sockets fecharem (teste com avanço de tempo).
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] verifyUpdateCodeSignature: false desativa a verificação de assinatura do auto-update

Labels sugeridas: security, severity:high, desktop, supply-chain

## Problema

`desktop/package.json` define `verifyUpdateCodeSignature: false` para o alvo Windows. O `electron-updater` (NSIS) deixa de conferir que o novo instalador foi assinado pelo mesmo publicador.

## Por que é explorável

A confiança no update passa a ser só "HTTPS + está no nosso GitHub Releases". Comprometimento do token/conta do GitHub, de um asset de release ou do pipeline => RCE persistente e silenciosa em todos os clientes Windows ("instalar ao sair"). `asar: false` piora (sem integridade de bundle).

## Evidência

```
`desktop/package.json:80-87`
```json
"win": { "target": ["nsis", "portable"], "icon": "icons/app-icon.png",
          "verifyUpdateCodeSignature": false }
...
`desktop/src/main/updates.ts:32-44` – `autoUpdater.checkForUpdatesAndNotify()`.
```

Impacto

RCE persistente em máquinas de usuários finais.

```

## Sugestão de correção
Assinar os builds Windows; remover `verifyUpdateCodeSignature: false`; habilitar `asar: true` + `asarIntegrity`.

## Critérios de aceite
- [ ] Build de release Windows assinado; flag não aparece como `false`.
- [ ] `latest.yml`/instalador adulterado é rejeitado pelo updater (teste manual documentado).
- [ ] `asar` habilitado.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] Handshake /ws aceita qualquer Origin (sem allowlist)

Labels sugeridas: security, severity:high, websocket

## Problema
`ws_handler` faz o upgrade sem checar o header `Origin`.

## Por que é explorável
Qualquer página web pode abrir `wss://.../ws` e falar todo o protocolo: criar salas, entrar em salas públicas,
cunhar credenciais TURN e conduzir os abusos de DoS/TURN. Não há cookies (então não é CSWSH clássico de
sessão), mas o OWASP WebSocket Cheat Sheet recomenda allowlist de `Origin` no handshake.

## Evidência
`crates/signaling/src/ws.rs:26-34`
```rust
pub async fn ws_handler(State(registry): State<Registry>, State(turn): State<Option<TurnConfig>>,
 headers: HeaderMap, ws: WebSocketUpgrade) -> impl IntoResponse {
 let client_key = client_key(&headers);
 ws.on_upgrade(move |socket| handle_socket(socket, registry, turn, client_key))
 // nenhuma checagem de Origin
}
```

## Impacto
Amplia a superfície de abuso para qualquer site que a vítima visite.

## Sugestão de correção
Validar `Origin` contra allowlist configurável (origem do app + `SCREEN_SHARE_URL`), recusar upgrade (403) caso
não bata. Defesa em profundidade, não autenticação.

## Critérios de aceite
- [ ] `Origin: https://evil.com` => 403, sem `101 Switching Protocols`.
- [ ] `Origin` da própria app => upgrade normal; e2e verde.
- [ ] Allowlist vem de env, com `const` documentando o default.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

Título: [Segurança] GET /api/rooms/:code sem auth nem rate limit expõe nome/ocupação e serve de oráculo de
enumeração

Labels sugeridas: security, severity:medium, idor, info-disclosure

## Problema
O endpoint devolve, para qualquer código existente e sem autenticação, o nome (escolhido por humano), a ocupação
e o flag de senha. Sem rate limit.

## Por que é explorável
Referência direta ao objeto (código) sem verificação de posse/membresia. O nome pode ser sensível ("Reunião
Diretoria – Fusão X"). Sem rate limit é um oráculo de existência que torna a enumeração observável; para sala
pública, adivinhar o código = entrar.

## Evidência
`crates/signaling/src/rooms_status.rs:7-24`
```rust
Some(summary) => Json(RoomStatus { exists: true, name: Some(summary.name),
 member_count: Some(summary.member_count), requires_password: Some(summary.requires_password) })),
...
`crates/signaling/src/registry.rs:488-498` – código: alfabeto de 31, comprimento 8 (~2^39,6).

Impacto
Divulgação de informação; reconhecimento/enumeração de salas; acesso a sala pública.

Sugestão de correção
Devolver apenas `{ exists, requires_password }` sem autenticação. Rate limit por IP. Opcional: mais entropia no
código de sala pública, ou separar `room_id` de `join_secret`.

Critérios de aceite
- [] Resposta não contém nome nem contagem de membros.

```



- [ ] Rate limit por IP (teste: N+1 chamadas em T s => 429).
- [ ] Fluxo de "sala não encontrada" na UI continua funcionando.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

Título: [Segurança] Sinalização Offer/Answer/IceCandidate aceita para qualquer co-membro sem relação de watch

Labels sugeridas: security, severity:medium, webrtc, privacy

#### ## Problema

O servidor relaya `Offer`/`Answer`/`IceCandidate`/`SetQuality` para qualquer `to` que seja membro da sala, sem exigir uma relação (sharer, viewer). O cliente responde a `ServerMessage::Offer` incondicionalmente.

#### ## Por que é explorável

Um membro malicioso envia `Offer { to: vítima }`; o cliente da vítima cria `RTCPeerConnection`, troca ICE (revelando IP de LAN e público ao atacante sem consentimento) e responde. Também permite spam de renegociação, injeção de candidatos e `QualityRequested` degradando o stream de terceiros.

#### ## Evidência

`crates/signaling/src/ws.rs:163-207`

```rust

```
ClientMessage::Offer { to, sdp } => {
    if let (Some(room), Some(from)) = (&room_code, &peer_id) {
        registry.relay(room, &to, ServerMessage::Offer { from: from.clone(), sdp });
    }
}
```

```

`apps/web/src/session/handler.rs:258-351` — cria PC, `ontrack` -> `

#### ## Impacto

Divulgação de IP entre co-membros sem consentimento; consumo forçado de recursos; perturbação de conexões P2P.

#### ## Sugestão de correção

- Servidor: só relayar quando existir `(sharer, viewer)` em `room.watchers` entre `from` e `to`.
- Cliente: ignorar `Offer` de `from` que o usuário não escolheu assistir e que não é espectador registrado.

#### ## Critérios de aceite

- [ ] `Offer` de A para B sem relação de watch é descartada pelo servidor (teste).
- [ ] Cliente não cria `RTCPeerConnection` para `Offer` de peer não relacionado.
- [ ] Cenário de watch legítimo (2 abas) continua estabelecendo mídia.

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

Título: [Segurança] Campos de protocolo sem validação (nick/room\_name/color/ReportLatency/WatchShare)

Labels sugeridas: security, severity:medium, input-validation

#### ## Problema

`nick`, `room\_name` e `color` não têm limite de tamanho nem sanitização de caracteres de controle/bidi, nem no cliente nem no servidor. `ReportLatency` aceita qualquer `u32`. `add\_watcher` aceita `sharer\_id` que não é membro/sharer.

#### ## Por que é explorável

Não é XSS (o Leptos escapa texto). Mas: um `nick` de vários MB é armazenado e \*\*retransmitido\*\* a cada membro (`PeerJoined` + snapshot) — amplificação; `room\_name` volta sem auth em `/api/rooms/:code`; `\n`/override RTL/homoglifos em `nick` renderizam na UI dos outros membros, permitindo passar-se por outro membro. `ReportLatency` falsifica o ping exibido de terceiros; `add\_watcher` polui `room.watchers`.

#### ## Evidência

`crates/protocol/src/client.rs:9-23`; `apps/web/src/features/home/create\_room.rs:43-59` (só `.trim()`);

`crates/signaling/src/registry.rs:109-160,311-327,349-366`; render em

`apps/web/src/features/room/member\_card.rs:249,278`.

#### ## Impacto

Amplificação de broadcast (DoS); falsificação de identidade visual; poluição de estado.

#### ## Sugestão de correção

No servidor (fonte da verdade) + espelho no cliente: `MAX\_NICK\_LEN` (~32), `MAX\_ROOM\_NAME\_LEN` (~64), `color` restrito à paleta (rejeitar), remover/recusar caracteres de controle e bidi, normalizar NFC. Teto para `ReportLatency`. `add\_watcher` só para membros.

#### ## Critérios de aceite

- [ ] `nick`/`room\_name` acima do limite => erro dedicado no `CreateRoom`/`JoinRoom`.
- [ ] `color` fora da paleta rejeitado no servidor.
- [ ] `nick` com `\n`/`u202E` recusado ou saneado (teste unitário).
- [ ] `ReportLatency` acima do teto e `WatchShare` para `sharer\_id` inexistente são descartados.

--- FIM ISSUE 8 ---

--- ISSUE 9 ---

Título: [Segurança] Limitador de brute force de senha confia em fly-client-ip sem validar proxy confiável

Labels sugeridas: security, severity:medium, auth, rate-limit

#### ## Problema

`client\_key` usa o header `fly-client-ip` diretamente como chave do balde de tentativas de senha, sem verificar que a requisição veio por um proxy confiável; sem o header, cai numa chave constante `unknown`.

#### ## Por que é explorável

Fora da Fly (ou com o app acessível diretamente), o atacante rotaciona `fly-client-ip` a cada conexão e ignora `MAX\_PASSWORD\_ATTEMPTS` => brute force ilimitado da senha da sala. Com o header ausente, 5 falhas de um atacante trancam todos os usuários daquela sala (DoS).

#### ## Evidência

```
`crates/signaling/src/ws.rs:12-24`
``rust
fn client_key(headers: &HeaderMap) -> String {
 headers.get("fly-client-ip").and_then(|v| v.to_str().ok())
 .map(str::to_owned).unwrap_or_else(|| "unknown".to_string())
}
```,  
`crates/signaling/src/registry.rs:429-445`.
```

Impacto

Brute force ilimitado de senha (fora da Fly); ou negação de entrada a usuários legítimos.

Sugestão de correção

Só confiar em `fly-client-ip`/`X-Forwarded-For` quando a conexão vier de `TRUSTED\_PROXIES`; senão usar `ConnectInfo<SocketAddr>`. Nunca cair numa chave constante compartilhada – usar o IP de peer real.

Critérios de aceite

- [] Variar `fly-client-ip` a cada request sem proxy confiável NÃO aumenta as tentativas permitidas (teste).
- [] Sem header e sem proxy confiável, a chave é o IP de peer real, não `unknown`.
- [] Config de proxies confiáveis documentada.

--- FIM ISSUE 9 ---

--- ISSUE 10 ---

Título: [Segurança] Desktop: renderer remoto sem bloqueio de navegação e pontes IPC sem checagem de origem

Labels sugeridas: security, severity:medium, desktop

Problema

A `BrowserWindow` principal carrega uma origem **remota** e não há `setWindowOpenHandler` nem handler de `will-navigate`/`will-redirect`. As pontes `desktopAudio`, `picker` e `desktopShare` são expostas a essa janela e nenhum handler `ipcMain` verifica `event.senderFrame`.

Por que é explorável

Um XSS/open-redirect na origem web (ou sequestro dela) navega o renderer privilegiado para conteúdo do atacante, que então chama:

- `picker.listAudioApps()` / `desktopAudio.start({mode:'screen',excludedBinaries:[]})` – enumera apps em execução e nomes de executáveis, e **inicia captura de áudio do sistema sem consentimento nem seletor**; no Windows o PCM misturado é empurrado ao renderer via `desktop-audio-pcm-chunk` (`platform/windows/audio.ts:26-29`) – captura encoberta exfiltrável.
- `desktopShare.linkReady(...)` – sobrescreve a área de transferência; `memberJoined` – notificação de SO com texto arbitrário.

Evidência

```
`desktop/src/main/window.ts:16-36`; `desktop/src/preload.ts:9-49`;  
`desktop/src/features/audio-share/ipc.ts:12-26`; `desktop/src/platform/windows/audio.ts:19-32`;  
`desktop/src/features/screen-share/quick-share.ts:11-32`.
```

Impacto

Escalada de XSS web para captura encoberta de áudio do sistema, enumeração de processos e sequestro de clipboard no app desktop.

Sugestão de correção

- `webContents.setWindowOpenHandler(() => ({ action: 'deny' }));` bloquear `will-navigate`/`will-redirect` para fora da origem do app.
- Fixar `contextIsolation`/`sandbox`/`nodeIntegration:false`/`webSecurity:true` explicitamente.
- Verificar `event.senderFrame` (origem == app) em todo handler `ipcMain`.
- Não emitir `desktop-audio-pcm-chunk` sem uma sessão de compartilhamento iniciada pelo app; validar o formato em `linkReady`.

Critérios de aceite

- [] `window.open`/navegação para `https://evil.com` do renderer são bloqueados (teste `\_electron`).
- [] Handlers IPC rejeitam `senderFrame` que não seja a origem do app (teste unitário).
- [] `desktopAudio.start` sem sessão ativa não emite PCM.

```
- [ ] `webPreferences` fixa as 4 flags.

--- FIM ISSUE 10 ---

--- ISSUE 11 ---

Título: [Segurança] App Axum sem cabeçalhos de segurança / CSP

Labels sugeridas: security, severity:low, hardening

## Problema
Nenhum middleware adiciona `Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options`, `Referrer-Policy` ou `Permissions-Policy`.

## Por que é relevante
Defesa em profundidade: uma CSP restritiva conteria um XSS hipotético e limitaria a superfície das pontes desktop (F10/F11); sem HSTS o `force_https` da Fly só redireciona; sem `Permissions-Policy` não há restrição declarada de `display-capture`/`camera`/`microphone`.

## Evidência
`apps/web/src/main.rs:39-56` — router sem `tower-http`. `apps/web/src/app.rs:14-39` — `shell` sem `

```

1. ``.github/workflows/ci-cd.yml`` usa ``superfly/flyctl-actions/setup-flyctl@master`` (sem pin) – o job tem acesso a ``secrets.FLY_API_TOKEN``.
2. ``desktop/package.json:52`` – ``asar: false`` (app sem integridade de bundle).
3. ``desktop/package.json:5-8`` – e-mail pessoal do mantenedor versionado (PII).

Impacto

Baixo/informativo: risco de cadeia de suprimento no deploy; adulteração local mais fácil; exposição de PII.

Sugestão de correção

1. Fixar todas as ``uses:`` de terceiros por SHA de commit.
2. ``asar: true`` + ``asarIntegrity``.
3. Usar um e-mail de projeto/alias.

Critérios de aceite

- [] Ações de Cit de terceiros fixadas por SHA.
- [] Build empacotado usa ``asar`` com verificação de integridade.
- [] ``author.email`` aponta para endereço de projeto.

--- FIM ISSUE 13 ---